Smart Complaint Management System

Software Architecture and Design Specification

Section 4 Software Design | Member 4
Hemanth V. | PES1UG25CS817
Team 8 | Version 1.0 | 11 October 2026

4 Design scope

We define the detailed design for the university complaint system using SRS Version 2.0. The design uses Python 3, Flask and SQLite, and preserves the Faculty-in-Charge, Grievance Committee and Dean hierarchy. Browser requests pass through server-side authentication and authorization before any protected operation.

4 1 Component interfaces

Component	Interface and responsibility
AuthService	registerStudent(input), login(input), requireActor(session), authorize(actor, action, complaint). Returns authenticated context or a typed authorization error.
ComplaintService	submitComplaint(actor, input), act(actor, id, action, expectedVersion), appeal(actor, id, input), listVisible(actor), recordFeedback(actor, id, input). Owns validation and transaction boundaries.
RoutingPolicy	initialLevel(severity), nextLevel(level), appealTarget(rejectedLevel). Pure functions centralize routing and reject escalation above Dean.
SQLite Repository	Read/write users, complaints, assignment history, appeals, audit entries, notifications and feedback. All mutation helpers accept the same transaction connection.

These are logical interfaces inside one Flask application. HTTP routes are defined in Sections 4 3 through 4 6; the browser cannot call repository methods directly. Separate auth, complaint, routing, audit and feedback modules behind these interfaces, sharing the transaction connection for mutations. Read-only dashboard and SLA handlers reuse the same access policy.

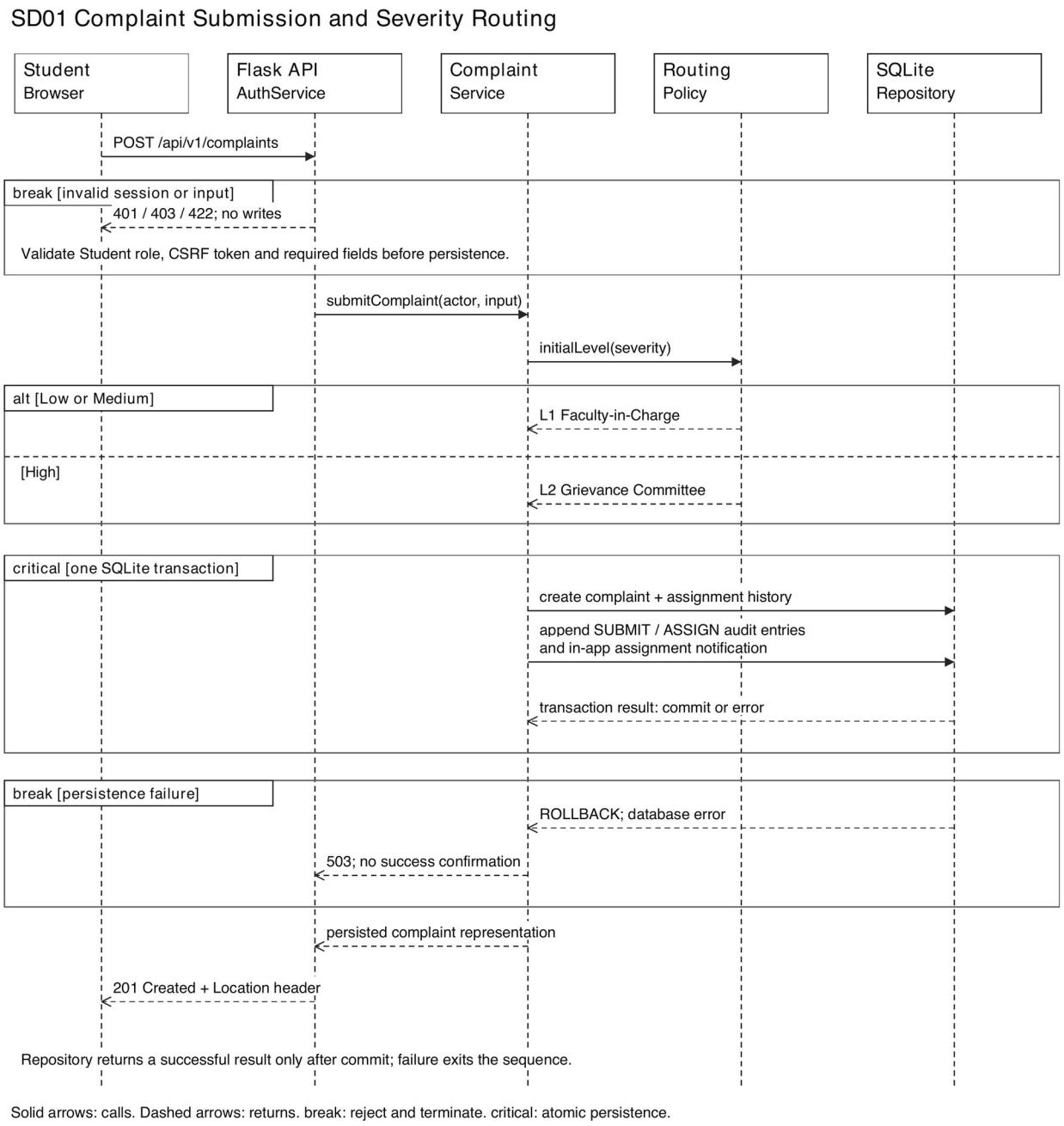
Design basis

Baseline: the team repository SRS.docx and SCMS_Test_Plan_2.0.docx, both treated as the current project references. Requirement identifiers are retained from the SRS feature sections. Module names and endpoint contracts are proposed design interfaces and must be checked against the final team architecture before sign-off. The Member 3 architecture document was not present in the repository at the time of this revision; this document therefore defines the Member 4 design and does not claim that architecture cross-check is complete.

4 2 Sequence diagrams

4 2 1 Complaint submission and automatic assignment

Precondition: authenticated Student. Input: category, description and Low/Medium/High severity. Success: a committed complaint with a unique ID, Submitted status, initial queue assignment, audit entries and an in-app notification. Covers REQ-2.1 to REQ-3.3 and REQ-6.1.

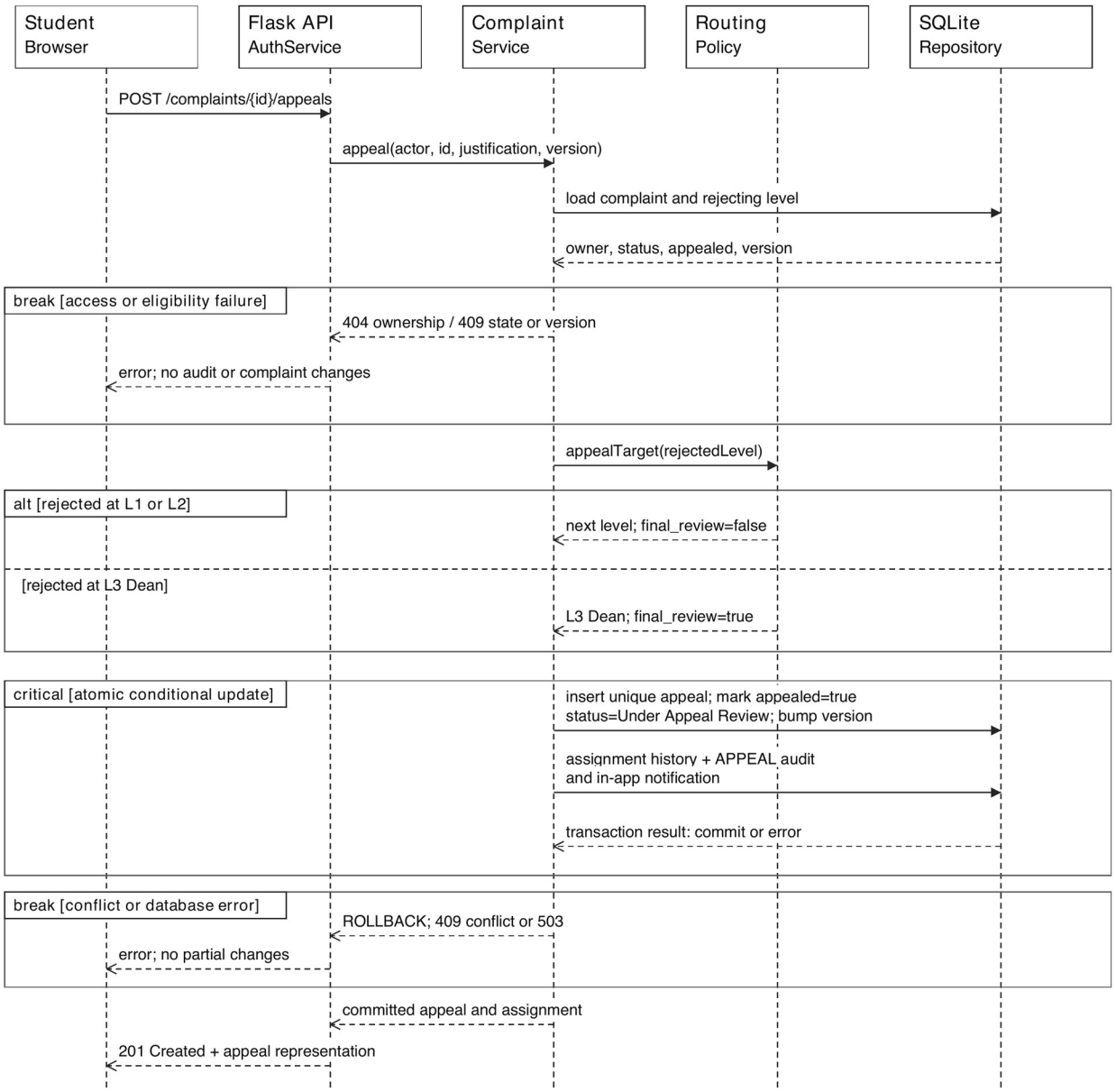


SD01. A validation failure terminates before writes. A persistence failure rolls back every write and returns an error. Successful confirmation follows commit, supporting NFR-REL-02. The detailed API is defined in Section 4 4.

4 2 2 Appeal and final review

Precondition: the submitting Student owns a Rejected complaint with no prior appeal. Success: exactly one appeal, Under Appeal Review status and a new assignment. L1 moves to L2; L2 moves to L3; a Dean rejection remains at L3 and is flagged for final review. Covers REQ-5.1 to REQ-5.4.

SD02 Single Appeal and Dean Final Review



Solid arrows: calls. Dashed arrows: returns. break: reject and terminate. critical: atomic persistence.

SD02. Conditional updates and a unique appeal constraint prevent concurrent duplicate appeals. Stale versions or an existing appeal return 409 with no partial state change. Authentication, CSRF and field validation run before the service call. See Section 4 5 for the request contract.

4 3 API conventions and authentication

Base path: /api/v1. Payloads use application/json; IDs are positive integers and timestamps are ISO 8601 UTC. Protected responses include Cache-Control: no-store. A signed cookie contains an opaque session ID, with Secure, HttpOnly and SameSite=Lax attributes. AuthService looks up the session and current user role in SQLite; logout deletes the session record. Mutations require X-CSRF-Token. GET /session supplies the token, including before login. Actor ID and role come from server-verified context.

Successful single-resource responses use {"data": {...}}. Collections use {"data": [...], "page": 1, "page_size": 20, "total": 3}. page starts at 1; page_size is 1 to 100. Reject unknown fields and blank required text. Common errors: 400 malformed JSON, 415 unsupported content type, 401 missing/expired session for protected routes, 403 wrong role or CSRF, and 500 unexpected failures. Endpoint-specific errors follow.

A01 Register student

POST /users | Public; CSRF required

Request: {"university_id":"PES1UG24CS579","name":"Example Student","email":"student@example.edu","password":"example-secret"}. University ID max 20, name/email max 100; valid email; nonempty password.

Response: 201; data contains user_id, university_id, name and role="Student". Role is always server assigned; password/hash is never returned.

Errors: 409 duplicate university ID/email; 422 invalid fields; 403 CSRF; 503 storage failure. Requirements: REQ-1.1, SEC-01.

A02 Authenticate

POST /session | Public; CSRF required

Request: {"university_id":"PES1UG24CS579","password":"example-secret"}.

Response: 200; set session cookie; data={user_id, role, csrf_token}. Replace the pre-login session and CSRF token.

Errors: 401 AUTHENTICATION_FAILED with "Invalid credentials." for both unknown ID and wrong password; 422 missing fields. Requirements: REQ-1.2, REQ-1.4, SEC-04.

A03 Session and logout

GET /session; DELETE /session | GET public; DELETE authenticated and CSRF protected

Request: GET has no body; DELETE has no body.

Response: GET 200: data={authenticated, user_id, role, csrf_token}; anonymous identity fields are null. DELETE 204: invalidate session and expire cookie.

Errors: DELETE 401 expired session; 403 CSRF. Requirements: REQ-1.2, REQ-1.3, SEC-02.

Passwords use a real salted, adaptive password hash supported by the Flask stack. The hash is verified by a dedicated library; Python hash() is not a password hash. Handler roles are provisioned by trusted deployment setup; registration exposes no role selection. No SSO or external authentication provider is needed.

4 4 Complaint submission and reading

C01 Create complaint

POST /complaints | Student; CSRF required

Request: {"category":"Facilities","description":"Classroom fan is not working.","severity":"Low"}. category 1-50 characters; description 1-1000; severity exactly Low, Medium or High. student_id, status and assignment are server generated.

Response: 201 + Location: /api/v1/complaints/101; data={complaint_id:101, status:"Submitted", severity:"Low", current_level:1, assigned_role:"Faculty-in-Charge", submitted_at, status_updated_at, version:1}.

Errors: 422 invalid fields/severity; 401 no session; 403 wrong role/CSRF; 503 failed transaction. Requirements: REQ-2.1 to REQ-2.3, REQ-3.1 to REQ-3.3, NFR-DATA-01 to NFR-DATA-03, NFR-REL-02.

C02 Complaint detail

GET /complaints/{id} | Owning Student or current/previous assigned handler level

Request: No body. Valid integer complaint ID.

Response: 200; data includes complaint_id, category, description, severity, status, current_level, assigned_role, submitted_at, status_updated_at, appealed, final_review, resolution_remarks, rejection_reason, version and allowed_actions. allowed_actions is a UI hint; the server rechecks every action.

Errors: 404 unknown/invisible complaint; 401 no session. Requirements: REQ-1.3, REQ-6.2, REQ-7.1, SEC-02.

C03 Visible dashboard

GET /complaints?page=1&page_size=20 | Authenticated

Request: Optional status filter must be a supported status; no body.

Response: 200 paginated summaries. Students see their own complaints; Faculty/Committee see their current queue. Dean sees complaints that reached L3, including L3 outcomes, and never complaints resolved at earlier levels. Handler actions remain limited to the current assignment.

Errors: 422 invalid filter/page; 401 no session. Requirements: REQ-7.1 to REQ-7.3, NFR-PERF-01, NFR-USAB-02.

Routing and persistence rules

Low and Medium route to L1 Faculty-in-Charge; High routes to L2 Grievance Committee. Create the complaint, initial assignment history, SUBMIT/ASSIGN audit records and queue notification in one transaction. Notifications are stored in-app for the assigned role queue; no email service is required. A complaint ID comes from the database primary key, not an in-memory counter.

New complaints remain Submitted after assignment. Assignment audit records may retain the same previous/new status while recording the changed level. Successful confirmation occurs only after commit. API payloads return role-appropriate projections rather than entire database rows.

4 5 Handling actions and appeals

Mutation payloads include expected_version, the version last read by the browser. ComplaintService validates authorization, legal state and version within the write transaction. It increments version once per accepted action. Proposed actionable states are Submitted, Escalated and Under Appeal Review; Resolved and Rejected are terminal until an eligible appeal reopens a rejection.

W01 Resolve

POST /complaints/{id}/resolve | Currently assigned handler level; CSRF required

Request: {"remarks":"Fan replaced and checked.","expected_version":1}. Trimmed remarks 1-500 characters.

Response: 200 updated complaint: status="Resolved", resolution_remarks stored, status_updated_at reset, version incremented; append RESOLVE audit.

Errors: 422 blank remarks; 403 wrong handler; 404 invisible/missing; 409 terminal state/stale version; 503 storage failure. Requirements: REQ-4.1, REQ-4.4, REQ-6.1.

W02 Reject

POST /complaints/{id}/reject | Currently assigned handler level; CSRF required

Request: {"reason":"Matter is outside this process.","expected_version":1}. Trimmed reason 1-500 characters.

Response: 200 updated complaint: status="Rejected", rejection_reason and rejected_level recorded; append REJECT audit.

Errors: 422 missing reason; 403 wrong handler; 404 invisible/missing; 409 invalid state/version; 503 storage failure. Requirements: REQ-4.2, REQ-4.4, REQ-5.2.

W03 Escalate

POST /complaints/{id}/escalate | Currently assigned handler level; CSRF required

Request: {"expected_version":1}. No client-selected target level.

Response: 200; L1 to L2 or L2 to L3, status="Escalated"; reset status_updated_at, record assignment/audit, notify the next queue.

Errors: 409 ESCALATION_LIMIT at L3, terminal state or stale version; 403 wrong handler; 404 invisible; 503 storage failure. Requirements: REQ-4.3, REQ-4.4, REQ-6.1.

W04 Appeal

POST /complaints/{id}/appeals | Owning Student; CSRF required

Request: {"justification":"Please reconsider the attached reasoning.","expected_version":2}. Trimmed justification 1-1000 characters (proposed design limit).

Response: 201; data={appeal_id, complaint_id, status:"Under Appeal Review", current_level, assigned_role, final_review, appealed:true, version}. Rejected L1 goes to L2; L2 to L3; L3 stays at Dean with final_review=true.

Errors: 409 APPEAL_ALREADY_USED ("Only one appeal is permitted."), NOT_REJECTED or STALE_VERSION; 422 justification; 404 not owner/missing; 503 storage failure. Requirements: REQ-5.1 to REQ-5.4, REQ-6.1.

4 6 Audit SLA notifications and feedback

Q01 Audit history

GET /complaints/{id}/audit | Owning Student or handler level with current/previous assignment

Request: No body; optional page/page_size.

Response: 200 chronological audit records: audit_id, actor_id, action, previous_status, new_status, timestamp, previous_level and new_level. Sort by timestamp then audit_id.

Errors: 404 invisible/missing; 401 no session. No POST/PATCH/DELETE audit endpoint exists. Requirements: REQ-6.1 to REQ-6.3, NFR-DATA-04, SEC-03.

Q02 Manual overdue check

POST /sla/check | Dean/Admin with explicit SLA_REVIEW permission; CSRF required

Request: {} uses configured threshold (default 7 days). Request cannot change the threshold.

Response: 200; data={checked_at, threshold_days, items:[{complaint_id, status, current_handler, days_overdue, can_escalate}]}. Compute days_overdue=max(0, elapsed_days-threshold_days); overdue means elapsed_days > threshold_days. No status mutation.

Errors: 403 lacks SLA_REVIEW/CSRF; 401 no session; 503 database unavailable. Requirements: REQ-8.1, REQ-8.2.

SLA design rule: inspect complaints whose time in the current status exceeds the configured threshold; exclude Resolved and Rejected complaints from escalation. The overdue review is a manually triggered, permission-protected view and is separate from the ordinary Dean dashboard. To satisfy REQ-8.2, a Dean/Admin with explicit SLA_REVIEW permission may invoke the one-click overdue escalation action through POST /sla/complaints/{id}/escalate with expected_version. This is a narrowly scoped SLA action, not ordinary handler authority: the service rechecks the SLA threshold, active state, current version and permission, then delegates the next-level decision to RoutingPolicy. The action may move an overdue L1 complaint to L2 or an overdue L2 complaint to L3; at L3 it returns 409 ESCALATION_LIMIT. It must write assignment history, audit and notification records atomically. REQ-4.4 remains enforced for normal Resolve/Reject/Escalate handler actions, and this SLA-specific permission does not grant those actions on another queue. No background scheduler runs; the overdue check is executed only on an authorized request.

Q03 In-app notifications

GET /notifications | Authenticated handler

Request: Optional page/page_size; no body.

Response: 200 notices for the current role queue: notification_id, complaint_id, event, created_at. Complaint access is rechecked on opening the notice.

Errors: 401 no session. Requirements: REQ-3.3.

Q04 Feedback

POST /complaints/{id}/feedback | Owning Student; CSRF required

Request: {"rating":4,"comment":"Issue was handled promptly."}; integer rating 1-5, comment optional up to 500 characters.

Response: 201; data={feedback_id, rating, submitted_at}. Proposed one-feedback-per-complaint rule prevents repeat prompts after submission.

Errors: 404 not owner/missing; 409 not Resolved or feedback already exists; 422 invalid rating/comment; 503 storage failure. Requirements: REQ-9.1, REQ-9.2.

GET /complaints/{id}/feedback returns {rating, comment, submitted_at} to an authorized handler; it excludes student ID, name and email and never joins them into the feedback projection. Owners can read their own feedback. This is identity suppression in the handler view, not a claim of unlinkability from complaint context. The owner sees a prompt on the first resolved detail view if feedback is absent.

4 7 Data design and state integrity

Entity	Keys and essential fields
users	user_id PK; university_id UNIQUE (20); email UNIQUE (100); name (100); password_hash (encoded string); role enum; created_at. Public registration always creates Student.
sessions	session_id opaque random PK; user_id nullable FK for pre-login sessions; CSRF token; expires_at. Rotate at login; delete at logout. Never accept identity/role from browser data.
complaints	complaint_id PK; student_id FK users; category (50); description (1000); severity enum; status enum; current_level 1-3; assigned_to nullable FK users; submitted_at; status_updated_at; remarks/reason (500); appealed bool; rejected_level; final_review bool; version >= 1.
assignment_history	assignment_id PK; complaint_id FK; level 1-3; actor_id FK; assigned_at. Keeps every level that held assignment, for historic audit visibility.
appeals	appeal_id PK; complaint_id UNIQUE FK; justification (1000); submitted_at; actor_id FK; rejected_level; target_level; final_review. UNIQUE enforces one appeal.
audit_logs	audit_id PK; complaint_id FK; actor_id FK; action; previous/new status; previous/new level; timestamp. Application exposes reads only; SQLite triggers reject UPDATE and DELETE.
notifications	notification_id PK; complaint_id FK; target_level; event; created_at. Created inside the same transaction as assignment.
feedback	feedback_id PK; complaint_id UNIQUE FK; rating CHECK 1-5; comment (500); submitted_at. No student_id in the handler-facing projection.
settings	key PK; value. sla_threshold_days defaults to 7 and must be positive. Configured by trusted deployment setup in this scope.

Atomic updates and validation

IDs/levels/version are SQLite INTEGER; text fields are TEXT; UTC times are ISO 8601 TEXT; booleans are INTEGER with CHECK IN (0,1). Lengths in parentheses follow SRS Appendix B; proposed appeal justification is limited to 1000 characters. Choose a password-hash encoding that fits the SRS 255-character field.

Enable foreign keys on every SQLite connection and enforce enum/length checks in the database as well as the API. Begin a short write transaction; read current state and authorization context; update WHERE complaint_id=? AND version=?; require one affected row; insert audit/history/notification records; commit. Appeal additionally requires status=Rejected and appealed=false and inserts the unique appeal row. Any failed step rolls back the entire transaction.

The assignment is a role queue, computed from current_level. The SRS assigned_to field can hold an optional named handler but does not replace role-level authorization. Preserve rejected_level when rejecting, so an appeal targets the rejecting level rather than guessing from the latest state. Resolved/Rejected do not allow handler mutations. A new eligible appeal changes Rejected to Under Appeal Review once.

Use parameterized SQL. Index complaints(student_id, submitted_at), complaints(current_level, status), complaints(status, status_updated_at), assignment_history(complaint_id, level), audit_logs(complaint_id, timestamp, audit_id) and notifications(target_level, created_at). Feedback uniqueness and appeal uniqueness are database constraints. Back up the database with a SQLite-aware method and verify recovery across a normal restart.

4 8 Error handling logging and monitoring

4 8 1 Consistent error responses

Error shape: {"error":{"code":"VALIDATION_ERROR","message":"Check the highlighted fields.","fields":{"description":"Description is required."},"request_id":"server-generated-id"}}. fields is present only for input errors. The UI retains valid form input, renders text safely and shows a reference ID for unexpected failures. Responses exclude stack traces, SQL and secrets.

HTTP	Code and handling
400 / 415	MALFORMED_JSON / UNSUPPORTED_MEDIA_TYPE. Reject before mutation.
401	AUTHENTICATION_REQUIRED or AUTHENTICATION_FAILED. Login failures always say Invalid credentials; expired sessions return to login.
403 / 404	FORBIDDEN / CSRF_FAILED for a permitted resource but disallowed action; NOT_FOUND for unknown or invisible complaint IDs to avoid disclosing existence.
409	STALE_VERSION, INVALID_STATE, ESCALATION_LIMIT, APPEAL_ALREADY_USED, DUPLICATE_ACCOUNT or FEEDBACK_ALREADY_EXISTS. Refresh state before resubmission.
422	VALIDATION_ERROR. Return field-specific messages; no complaint/audit writes.
503	PERSISTENCE_UNAVAILABLE. Roll back; show no success message. A committed response lost in transit requires checking the dashboard before retrying creation.
500	INTERNAL_ERROR. Roll back; log a redacted exception with request ID; return a generic message.

4 8 2 Logging and operational checks

Separate immutable business audit records from operational logs. Business audit stores actor, action, old/new state and UTC time for accepted complaint actions; denied requests belong in operational security logs. No application user can modify or delete audit rows. Database-owner access remains a deployment trust boundary.

Operational JSON logs contain request_id, timestamp, route template, method, status, elapsed_ms, component, error_code and optional internal actor/complaint IDs. Never log passwords, password hashes, cookies, CSRF tokens, raw bodies, complaint descriptions, appeal text or feedback comments. Restrict log access and set rotation/retention through deployment configuration after university policy is agreed.

Measure complaint submission and dashboard latency, 5xx rate, denied operations, SQLite lock/busy failures and rollback counts. Validate submission <=3 s and dashboard <=2 s under a documented mini-project workload; the SRS does not quantify that workload. Record actual results only after testing. A manual readiness check executes a lightweight DB query and verifies the configured database is available; it exposes no records. SLA evaluation occurs only on an authorized user request.

4 9 Requirement traceability

This mapping preserves the functional, security and nonfunctional IDs in the SRS feature sections. References are design sections and endpoint identifiers, not claims of implemented or executed functionality.

SRS requirements	Design reference
REQ-1.1 to REQ-1.4; SEC-01, SEC-02, SEC-04	4 3 A01-A03; 4 8. Student-only registration, session/RBAC, real hashing and generic login failures.
REQ-2.1 to REQ-2.3	4 2 1 SD01; 4 4 C01; 4 7. Field validation, ID and UTC timestamp.
REQ-3.1 to REQ-3.3	4 2 1 SD01; 4 4 routing; 4 6 Q03. Severity routing and in-app notice.
REQ-4.1 to REQ-4.4	4 5 W01-W03; 4 7. Assigned-level authorization, remarks/reason and Dean ceiling.
REQ-5.1 to REQ-5.4	4 2 2 SD02; 4 5 W04; 4 7. Unique appeal, next level and Dean final review.
REQ-6.1 to REQ-6.3; SEC-03	4 6 Q01; 4 7 audit/assignment history; 4 8. Atomic audit, historic visibility, no edits/deletes.
REQ-7.1 to REQ-7.3	4 4 C02-C03. Owner/current queue filtering and Dean reached-L3 view.
REQ-8.1 to REQ-8.2	4 6 Q02 and overdue escalation. Manual check, threshold, days overdue and escalation ceiling.
REQ-9.1 to REQ-9.2	4 6 Q04 and feedback read. Resolved-only prompt and identity suppression.
NFR-PERF-01, NFR-PERF-02	4 7 indexes; 4 8 timings and workload measurement. Targets 2 s and 3 s respectively.
NFR-DATA-01 to NFR-DATA-04	4 4 C01; 4 7 transaction, unique ID, no invalid writes, ordered immutable audit.
NFR-USAB-01, NFR-USAB-02	4 4 allowed_actions/status; 4 8 field errors. UI labels and status badges accompany textual status.
NFR-MAIN-01	4 1 interfaces; 4 7 centralized transition policy. Separate auth, complaints, routing, audit and feedback modules.
NFR-REL-01, NFR-REL-02	4 7 file persistence and backup; 4 2 1 commit-before-confirmation; 4 8 rollback handling.
SO-01, SO-02; BR-01 to BR-08	4 3 authorization; 4 4-4 7 workflows; 4 8 audit/accountability. Business rules are preserved by these contracts.

Source references

Team 8 repository: https://github.com/SE-MiniProject-Team-8/Smart-Complaint-Management-System
SRS.docx Version 2.0 and SCMS_Test_Plan_2.0.docx Version 2.0, retrieved 3 October 2026.
Member allocation: deliverable-2-workflow, Member 4 Software Design.